Day 5 — RTOS Fundamentals

Real-Time Operating Systems with FreeRTOS

1 | What Is an RTOS?

A Real-Time Operating System (RTOS) schedules tasks with deterministic timing guarantees. Unlike a general-purpose OS, an RTOS ensures that critical tasks meet hard deadlines — essential in embedded control, communications, and safety systems.

Bare-metal	Single super-loop; no preemption; simple but fragile under load.
Cooperative OS	Tasks yield voluntarily; still non-preemptive; better structure.
Preemptive RTOS	Scheduler interrupts any task at tick; highest-priority task always runs.
Hard Real-Time	Missing a deadline = system failure (e.g. airbag controller).
Soft Real-Time	Missed deadline degrades quality but is tolerable (e.g. audio stream).

2 | FreeRTOS Overview

FreeRTOS is the most widely deployed open-source RTOS, supporting 40+ MCU architectures. It ships as a small C library you add to your project. Key files:

tasks.c / .h	Task creation, scheduler, context switch.
queue.c / .h	Thread-safe FIFO queues & mailboxes.
semphr.h	Semaphores and mutexes (built on queues).
timers.c / .h	Software timers (one-shot or periodic).
FreeRTOSConfig.h	Project-specific configuration — heap size, tick rate, priorities.
portable/	Architecture port layer (context switch assembly).

3 | Tasks

Every unit of work in FreeRTOS is a **task** — an infinite loop running in its own stack context. The scheduler selects which task to run based on priority.

Task states:

State	Description	Transition
Running	Currently executing on the CPU.	Preempted → Ready; blocks → Blocked
Ready	Eligible to run; waiting for CPU.	Scheduler picks it → Running
Blocked	Waiting for event/delay/queue.	Event occurs → Ready
Suspended	Excluded from scheduling (vTaskSuspend).	vTaskResume → Ready
Deleted	Resources freed; task gone.	—

Core API:

```
xTaskCreate( vMyTask, "MyTask", 256, NULL, 2, &xHandle; );
vTaskDelay( pdMS_TO_TICKS(100) ); // block for 100 ms
vTaskDelayUntil( &xLast;, pdMS_TO_TICKS(50) ); // precise period
vTaskDelete( xHandle ); // NULL = self-delete
```

4 | Scheduler & Priorities

Priority range	0 (lowest / idle) to configMAX_PRIORITIES-1 (highest).
Preemption	Higher-priority ready task immediately preempts lower.
Round-robin	Equal-priority tasks share CPU in time slices (1 tick each).
Idle task	Priority 0; always runs when nothing else is ready; runs hooks.
Tick rate	configTICK_RATE_HZ (e.g. 1000 Hz = 1 ms resolution).
Time-slicing	Enabled with configUSE_TIME_SLICING 1 in FreeRTOSConfig.h.

5 | Queues

Queues provide thread-safe FIFO data passing between tasks or ISRs. They copy data by value (not by reference), making them inherently safe.

```
xQueue = xQueueCreate( 10, sizeof(uint32_t) );
xQueueSend( xQueue, &value;, pdMS_TO_TICKS(10) ); // producer
xQueueReceive( xQueue, &rxVal;, portMAX_DELAY ); // consumer
xQueueSendFromISR( xQueue, &val;, &xHigherPriorityTaskWoken; );
```

■ *Always call portYIELD_FROM_ISR(xHigherPriorityTaskWoken) at the end of any ISR that uses a FreeRTOS API.*

6 | Semaphores & Mutexes

Type	API	Use-case
Binary semaphore	xSemaphoreCreateBinary()	ISR → task signalling.
Counting semaphore	xSemaphoreCreateCounting(max,init)	Resource pools, event counters.
Mutex	xSemaphoreCreateMutex()	Mutual exclusion; priority inheritance built in.
Recursive mutex	xSemaphoreCreateRecursiveMutex()	Re-entrant code paths.

Priority Inversion: a low-priority task holds a mutex needed by a high-priority task. FreeRTOS mutexes implement **priority inheritance** to reduce this window automatically.

7 | Software Timers

Software timers execute a callback in the context of the Timer Service Task (daemon task). They require **configUSE_TIMERS 1** in FreeRTOSConfig.h.

```
xTimer = xTimerCreate("LED", pdMS_TO_TICKS(500), pdTRUE, 0, vLEDCallback);
```

```
xTimerStart( xTimer, 0 );
xTimerStop( xTimer, 0 );
```

One-shot (pdFALSE)	Fires once then stops; restart manually.
Auto-reload (pdTRUE)	Fires repeatedly at set period.
Resolution	1 tick; not suitable for sub-tick precision.
Callback context	Runs in timer daemon, NOT in ISR — can use most FreeRTOS APIs.

8 | Memory Management

Heap Scheme	Description	Best for
heap_1.c	Simple bump allocator — no free().	Static-only apps.
heap_2.c	Best-fit; allows free(); no coalescing.	Fixed-size allocations.
heap_3.c	Wraps stdlib malloc/free; thread-safe wrapper.	When libc heap is available.
heap_4.c	First-fit with coalescing; most common choice.	General embedded use.
heap_5.c	Like heap_4 but spans multiple memory regions.	Chips with SRAM + CCMRAM.

Monitor heap usage with xPortGetFreeHeapSize() and xPortGetMinimumEverFreeHeapSize().

9 | FreeRTOSConfig.h — Key Defines

Define	Typical Value	Effect
configCPU_CLOCK_HZ	168000000UL	CPU frequency for tick calculation.
configTICK_RATE_HZ	1000	RTOS tick = 1 ms.
configMAX_PRIORITIES	8	Number of priority levels.
configMINIMAL_STACK_SIZE	128	Idle task stack (words).
configTOTAL_HEAP_SIZE	(32*1024)	FreeRTOS heap pool size.
configUSE_PREEMPTION	1	Enable preemptive scheduler.
configUSE_MUTEXES	1	Enable mutex support.
configUSE_TIMERS	1	Enable software timers.
INCLUDE_vTaskDelete	1	Allow task deletion.

10 | Common Pitfalls & Debug Tips

Stack overflow	Increase stack size or enable configCHECK_FOR_STACK_OVERFLOW (mode 2).
Heap exhaustion	Call xPortGetMinimumEverFreeHeapSize() after init to check headroom.

Priority inversion	Use mutexes (not binary semaphores) for shared resource protection.
Blocking in ISR	Never call vTaskDelay or non-FromISR APIs inside an interrupt.
Forgetting portEND_SWITCHING_ISR	Always yield from ISR if a higher-priority task was unblocked.
Floating-point in tasks	Save FPU context: configUSE_TASK_FPU_SUPPORT must be enabled.
Deadlock	Acquire multiple mutexes in consistent order across all tasks.

7-Day Course Roadmap:

Day	Topic
Day 1 ✓	Introduction — Architecture, GPIO, Memory, Protocols
Day 2 →	Interrupts, Timers & PWM
Day 3	UART, SPI, I2C — Deep Dive
Day 4	ADC/DAC & Sensor Interfacing
Day 6	Power Management & Low-Power Design
Day 7	Bootloaders, OTA & System Design Project
